



Machine Learning 1

Lecture 7 - Classification With Basis Functions
- Probabilistic Discriminative Models - Logistic
Regression

Erik Bekkers

Catch-up:

Classification with linear reg

The perceptron

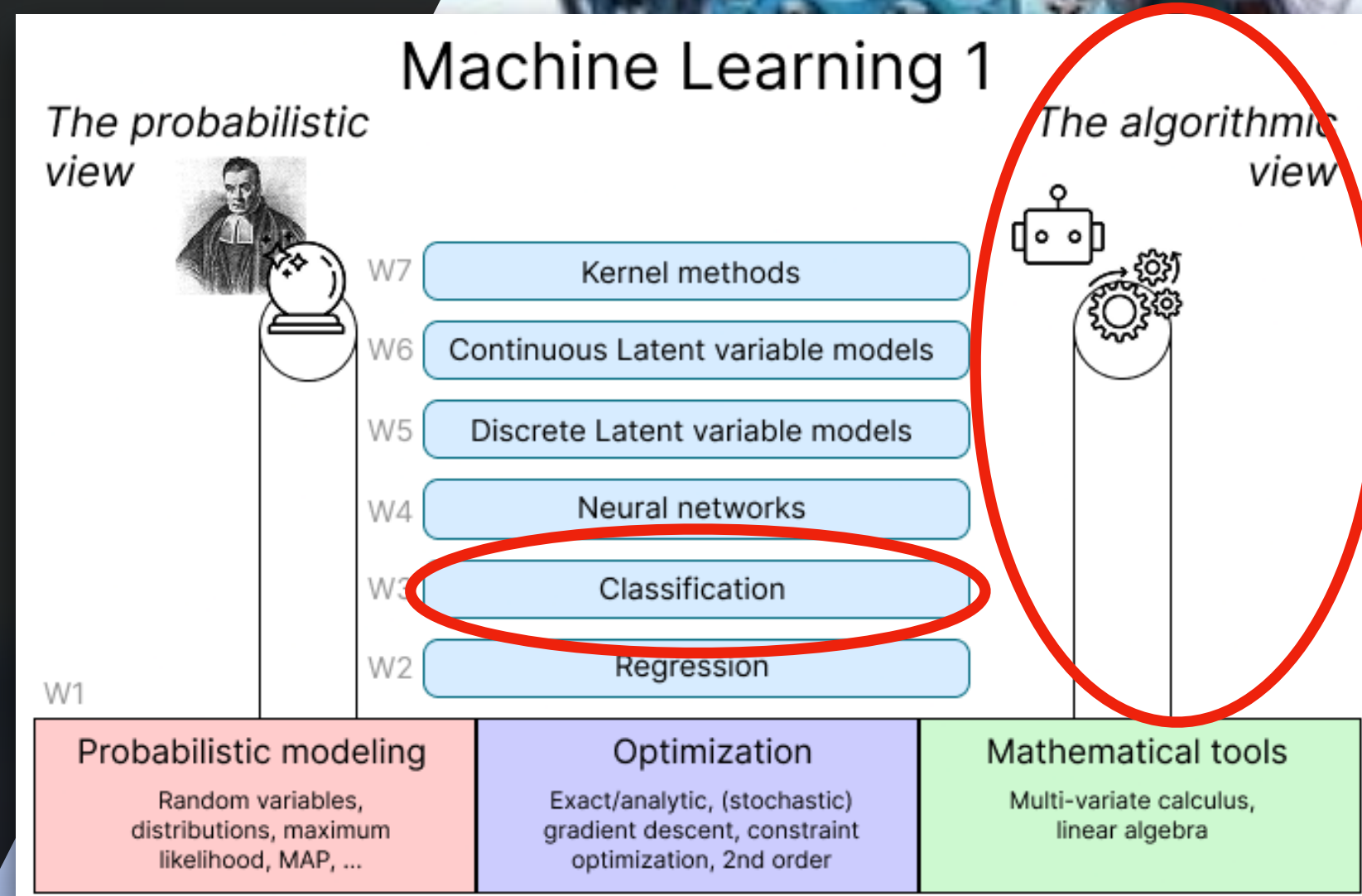


Machine Learning 1

Lecture 6.3 - Supervised Learning
Classification - **Discriminative Models**

Erik Bekkers

(Bishop 4.0, 4.1.1, 4.1.2)



Classification Strategies

- Discriminant functions

Direct mapping of input to target:

$$t = \underline{y(\mathbf{x}, \mathbf{w})}$$

- Probabilistic discriminative models

Posterior class probabilities:

$$p(C_k | \mathbf{x}) = \sigma(a(\mathbf{x}))$$

logistic sigmoid or softmax

- Probabilistic generative models

Class-conditional densities:

means we can sample from the learned data distribution

Prior class probabilities:

LDA

$$p(C_k)$$

Discriminant Functions: Two Classes

- ▶ Input: $\mathbf{x} \in \mathbb{R}^D$
- ▶ Targets: $t \in \{C_1, C_2\} = t=0, t=1$

Discriminant functions:

- ▶ Direct mapping of input to target

We now consider generalized Linear Models (GLM)

- ▶ $y(\mathbf{x}, \tilde{\mathbf{w}}) = f(\tilde{\mathbf{w}}^T \boldsymbol{\phi}(\mathbf{x}))$
- ▶ with $\boldsymbol{\phi}(\mathbf{x}) = (1, \phi_1(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))^T$
- ▶ Decision boundary: $y(\mathbf{x}, \tilde{\mathbf{w}}) = \text{const}$

"so, why not directly model such a y ?"

Discriminant Functions: Two Classes

- Simplest discriminant function ($f(\mathbf{x}) = x$, canonical basis $\phi_0(\mathbf{x}) = 1$, $\phi_j(\mathbf{x}) = x_j$, $j = 1, \dots, D$)

$$y(\mathbf{x}, \tilde{\mathbf{w}}) = \mathbf{w}^T \mathbf{x} + w_0$$

- Decision boundary: $y(\mathbf{x}, \tilde{\mathbf{w}}) = 0$

$$\underline{w}^T \underline{x} + w_0 = 0$$

- Consider 2 datapoints $\mathbf{x}_A$ and $\mathbf{x}_B$ on decision boundary

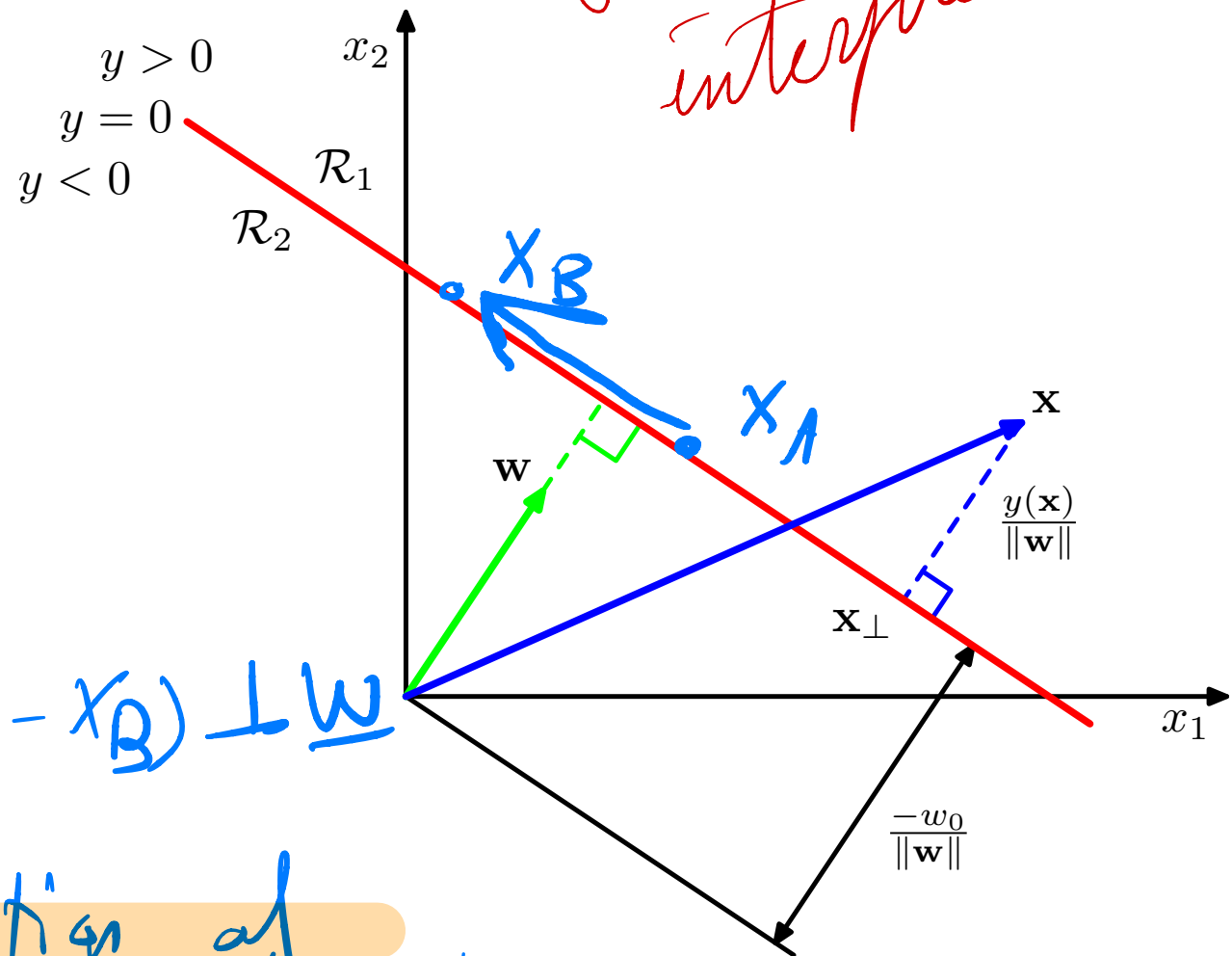
$$\underline{w}^T x_A + w_0 = \underline{w}^T x_B + w_0$$

$$y(\mathbf{x}_A) = y(\mathbf{x}_B) = 0$$

$$\underline{w}^T (x_B - x_A) = 0$$

Still usefull to establish
some intuition.

Geometric interpretation



i.e.,

$$(x_A - x_B) \perp \underline{w}$$

w determines orientation of decision boundary

"normal vector of the hyperplane"

Figure: 2-class decision boundary

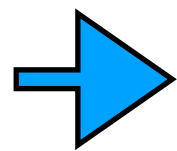
Discriminant Functions: Two Classes

- Take $\mathbf{x}'$ a point on a decision surface: $y(\mathbf{x}') = 0$

$$\mathbf{w}^T \mathbf{x}' + w_0 = 0 \Leftrightarrow \mathbf{w}^T \mathbf{x}' = -w_0$$

- Normal (signed) distance d from origin to decision surface

$$d = \frac{\mathbf{w}^T \mathbf{x}'}{\|\mathbf{w}\|} = \frac{-w_0}{\|\mathbf{w}\|}$$



$\mathbf{w}$ shifts the d.b. away from the origin

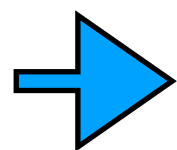
- Normal (signed) distances r from general $\mathbf{x}$ to decision surface:

Let $\mathbf{x} = \mathbf{x}_\perp + r \frac{\mathbf{w}}{\|\mathbf{w}\|}$

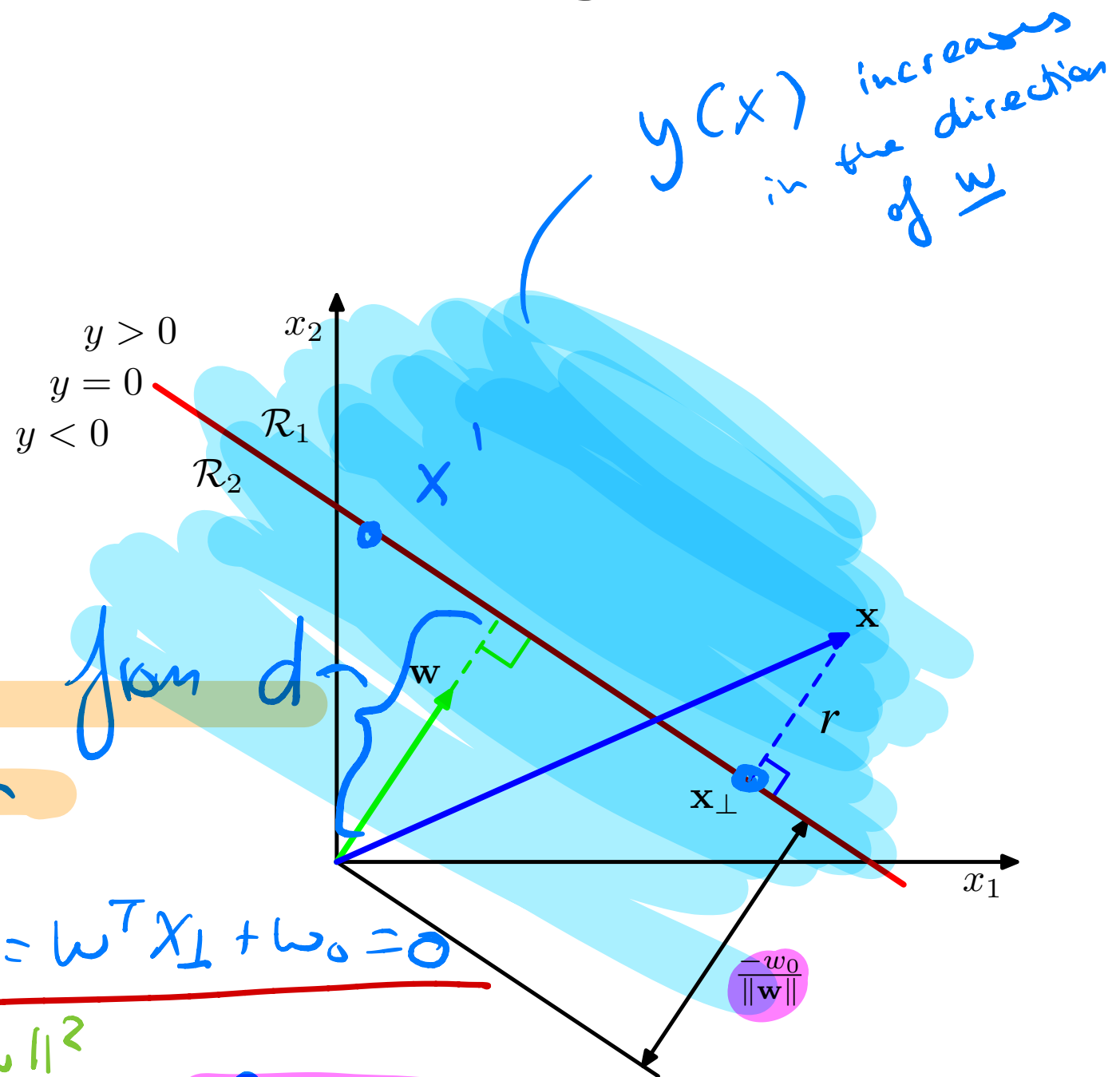
Then, $y(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + w_0 = \mathbf{w}^T \mathbf{x}_\perp + r \frac{\mathbf{w}^T \mathbf{w}}{\|\mathbf{w}\|} + w_0 = r \cdot \|\mathbf{w}\|$

$y(\mathbf{x}_\perp) = \mathbf{w}^T \mathbf{x}_\perp + w_0 = 0$

$y(\mathbf{x}_\perp) = 0$



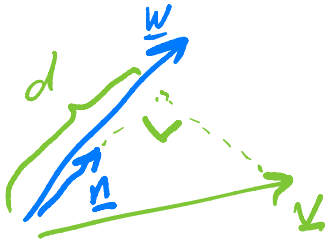
Distance r of a point to the d.b. is $r = \frac{y(\mathbf{x})}{\|\mathbf{w}\|}$



Orthogonal projection

$\underline{w}$ a vector

$\underline{n} = \frac{\underline{w}}{\|\underline{w}\|}$ a normalized vector



the orthogonal projection of $\underline{v}$ on $\underline{n}$
is given by

$$d = \underline{n}^T \underline{v}$$

More about this in the PCA lecture

So the distance
of a point x to
the decision boundary
is proportional to $y(\underline{x})$

$$r = \frac{y(x)}{\|w\|}$$

Discriminant Functions: Multiple Classes

- ▶ K -class discriminant

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

like assigning to max prob

- ▶ Assign $\mathbf{x}$ to C_k if $\forall_{j \neq k} : y_k(\mathbf{x}) > y_j(\mathbf{x})$

- ▶ Decision boundary between R_k and R_j :

$$y_k(\mathbf{x}) = y_j(\mathbf{x}) \iff y_k(\mathbf{x}) - y_j(\mathbf{x}) = 0$$

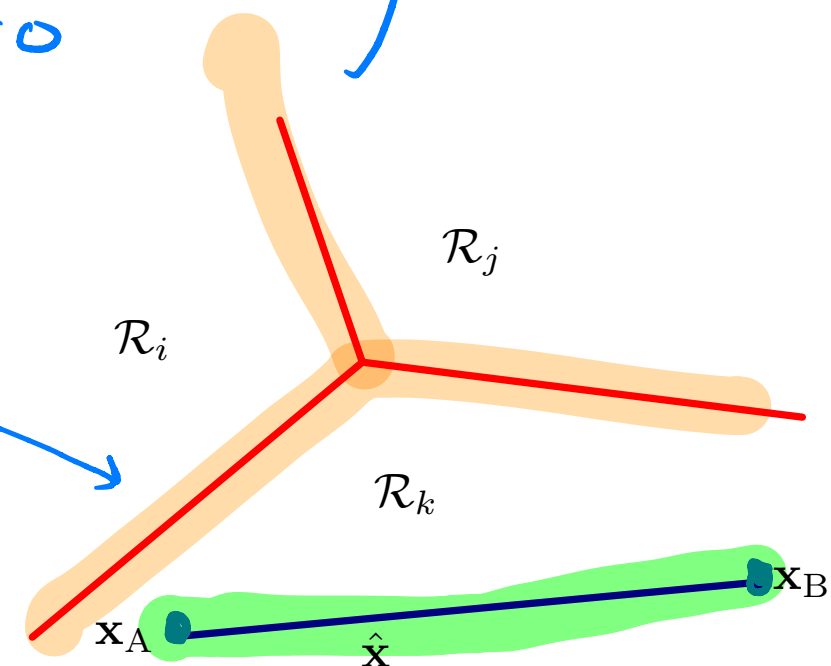
linear d.b

$\iff$

$$(\underline{w}_k - \underline{w}_j) \underline{x} + (w_{k0} - w_{j0}) = 0$$

Thus linear

- ▶ Decision regions (for GLM) are convex

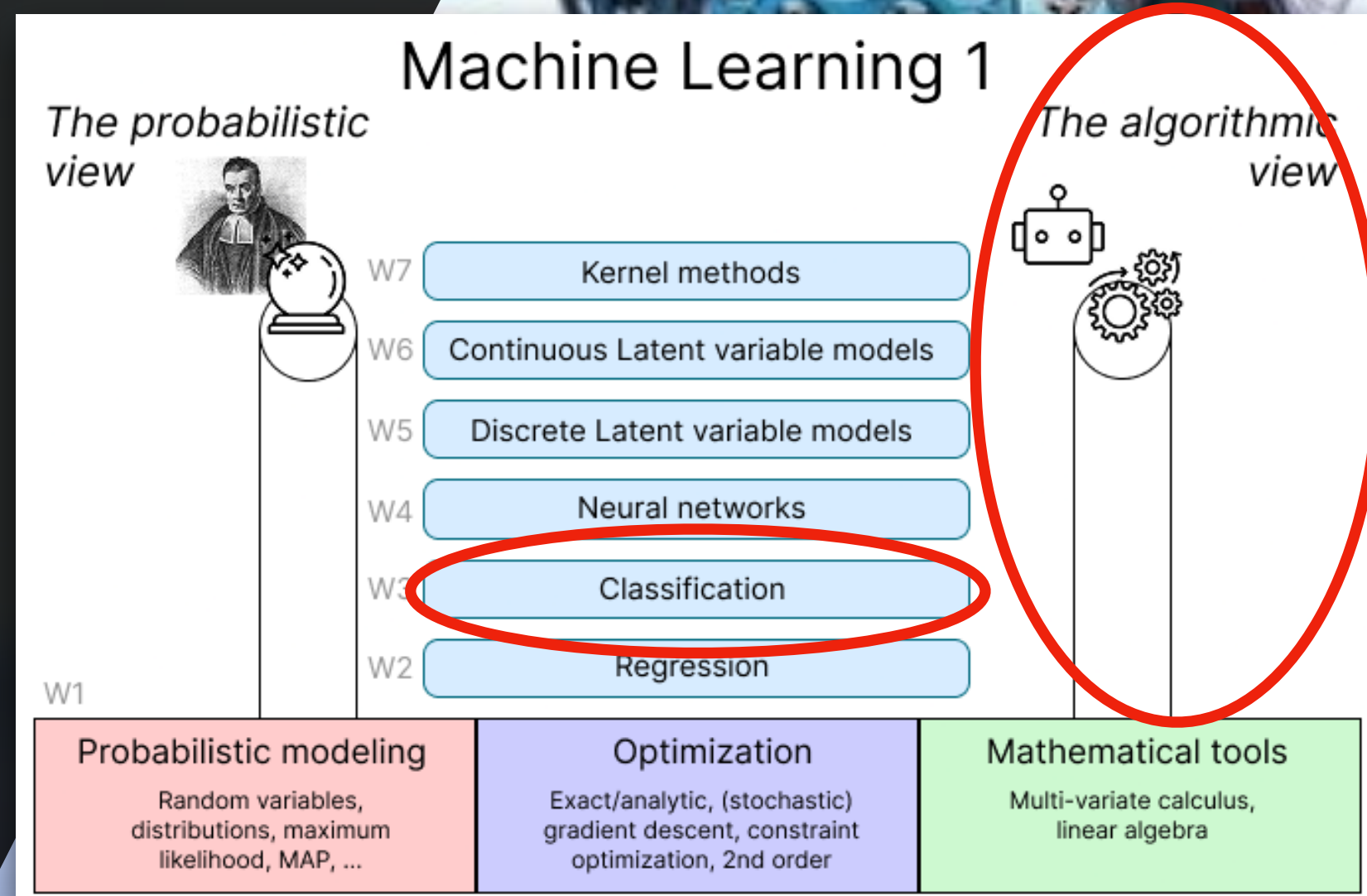


Machine Learning 1

Lecture 6.4 - Supervised Learning
Classification? - Discriminative Models - **Least Squares Regression**

Erik Bekkers

(Bishop 4.1.3)



Least Squares for Classification

- Each class C_k has its own linear model:

$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

- Shorter notation: $y(\mathbf{x}) = \tilde{\mathbf{W}}^T \tilde{\mathbf{x}}$

- Matrix $\tilde{\mathbf{W}}$, column k contains:

$$\tilde{w}_k = \text{Concat}(w_{k0}, \mathbf{w}_k) \in \mathbb{R}^M$$

- Input vector:

$$\tilde{\mathbf{x}} = \text{Concat}(1, \mathbf{x}) \in \mathbb{R}^M$$

- Output/prediction vector:

$$\mathbf{y}(\mathbf{x}) = \begin{pmatrix} y_1(\mathbf{x}) \\ y_2(\mathbf{x}) \\ \vdots \\ y_K(\mathbf{x}) \end{pmatrix} = \tilde{\mathbf{W}}^T \tilde{\mathbf{x}} \in \mathbb{R}^K$$

- Classification:** Assign $\mathbf{x}$ to class C_K if $k = \underset{j}{\operatorname{argmax}} y_j(\mathbf{x})$

Discriminative linear model

Least Squares for Classification (II)

- ▶ Data set: $N \times (D + 1)$ data matrix $N \times K$ target matrix

$$\tilde{\mathbf{X}} = \begin{pmatrix} \tilde{\mathbf{x}}_1^T \\ \vdots \\ \tilde{\mathbf{x}}_N^T \end{pmatrix}$$

$$\mathbf{T} = \begin{pmatrix} \mathbf{t}_1^T \\ \vdots \\ \mathbf{t}_N^T \end{pmatrix}$$

one-hot encoding for class C_3

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix}$$

$y_k(x_n)$

- ▶ Use sum-of-squares regression error function

$$\sum_n \sum_k (\tilde{\mathbf{w}}_k^T \tilde{\mathbf{x}}_n - t_{nk})^2$$

$$E_D(\tilde{\mathbf{W}}) = \frac{1}{2} \text{Tr} [(\tilde{\mathbf{X}}\tilde{\mathbf{W}} - \mathbf{T})^T (\tilde{\mathbf{X}}\tilde{\mathbf{W}} - \mathbf{T})] = \frac{1}{2} \sum_n \sum_k \left(\sum_m \tilde{x}_{nm} \tilde{w}_{mk} - t_{nk} \right)^2$$

- ▶ Minimize $E_D(\tilde{\mathbf{W}})$ as a function of $\tilde{\mathbf{W}}$: $\frac{\partial E}{\partial \mathbf{W}} = 0$

- ▶ Solution: $\tilde{\mathbf{W}}_{\text{LS}} = (\tilde{\mathbf{X}}^T \tilde{\mathbf{X}})^{-1} \tilde{\mathbf{X}}^T \mathbf{T} = \tilde{\mathbf{X}}^\dagger \mathbf{T}$

- ▶ Discriminant function: $\mathbf{y}_{\text{LS}}(\mathbf{x}) = \tilde{\mathbf{w}}_{\text{LS}}^T \tilde{\mathbf{x}} \in \mathbb{R}^k$

Matrix contains all products $\text{error}_i \cdot \text{error}_j$ but we are only interested in the trace (sum of diagonal) which is $\sum_k \text{error}_k^2$

$W: M \times K$, each column is one $\underline{w}_k$
 $W = (\underline{w}_1, \underline{w}_2, \dots, \underline{w}_K)$

$X: N \times M$, each row is an data point

$$X = \begin{pmatrix} \underline{x}_1^T \\ \underline{x}_2^T \\ \vdots \end{pmatrix}$$

$XW: N \times K$, K predictions per data point

$$[XW]_{nk} = \sum_{m=0}^{M-1} X_{nm} W_{mk} = \underline{x}_n^T \underline{w}_k = \underline{w}_k^T \underline{x}_n = y_k(\underline{x}_n)$$

$$[XW - T]_{nk} = y_k(\underline{x}_n) - t_{nk}$$

$$(XW - T)^T (XW - T): K \times K$$

$$[(XW - T)^T (XW - T)]_{kk'} = \sum_n (y_k(\underline{x}_n) - t_{nk}) (y_{k'}(\underline{x}_n) - t_{nk'})$$

we're only interested in the terms $k=k'$

The Trace Tr operator takes sum along diagonal

$$\text{Tr}(\text{mat } A) = \sum_i A_{ii}$$

for a matrix M with elements m_{ij} , i.e., $M = \begin{pmatrix} m_{11} & m_{12} & m_{13} & \dots \\ m_{21} & m_{22} & m_{23} & \dots \\ m_{31} & m_{32} & m_{33} & \dots \\ \vdots & \vdots & \vdots & \ddots \end{pmatrix}$

the trace $\text{Tr}[M]$ sums over the diagonal.

On the previous slide the matrix is $K \times K$ and contains the errors

$$m_{ij} = \sum_n (y_{ni} - t_{ni})(t_{nj} - t_{nj})$$

whereas we're only interested in the diagonal

$$m_{kk} = \sum_n (y_{nk} - t_{nk})^2$$

Least Squares for Classification: Problems

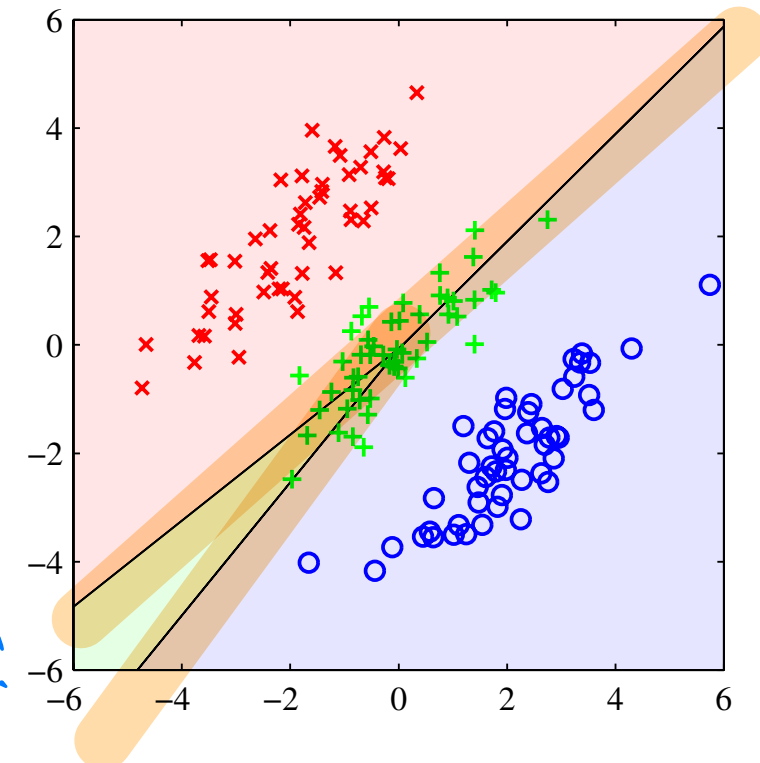
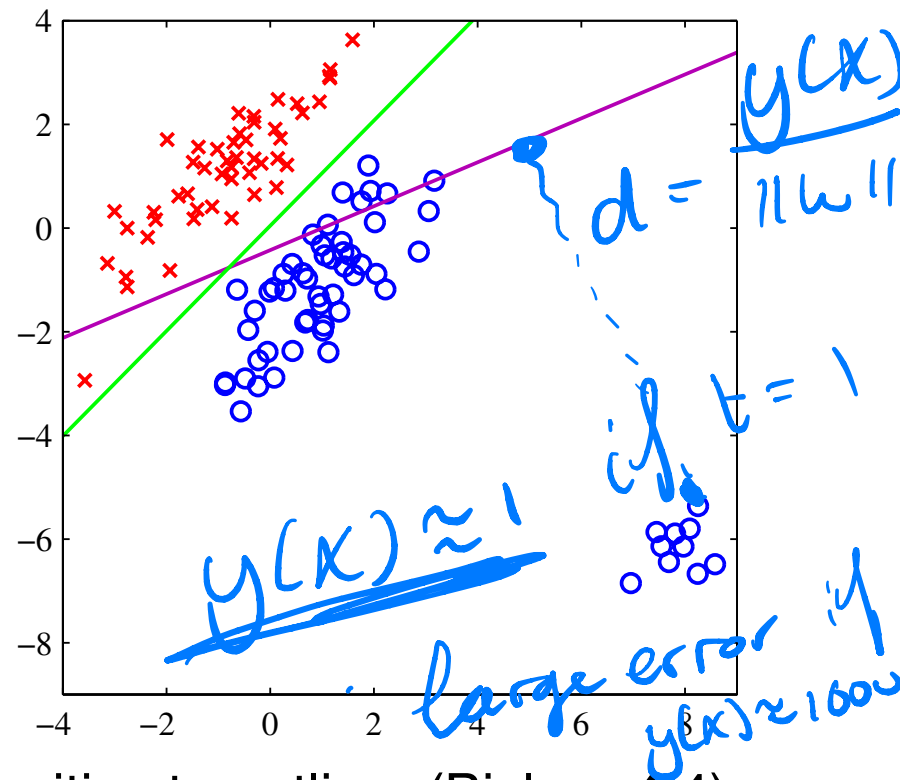
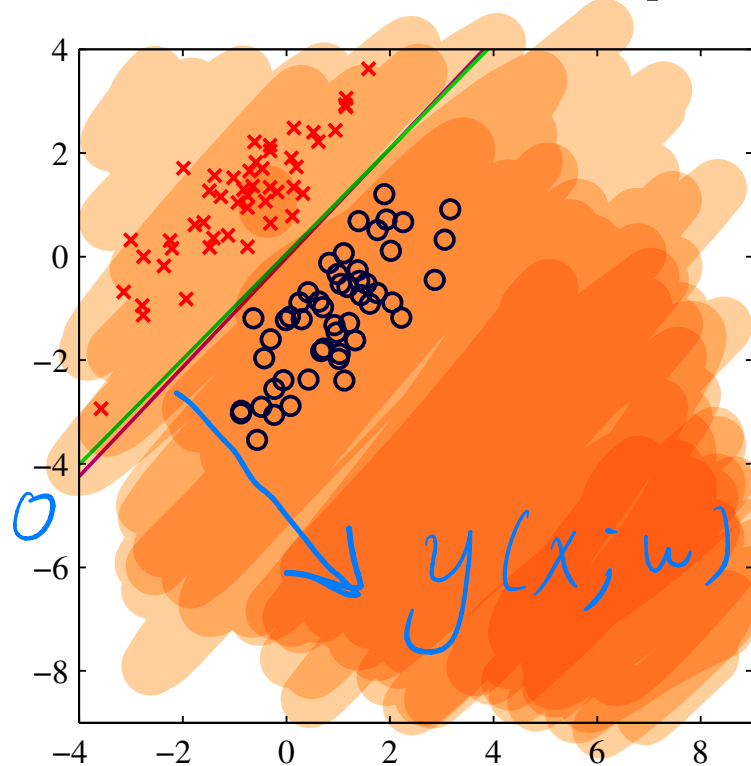


Figure: least squares is very sensitive to outliers (Bishop 4.4)

Figure: masking for least squares for $K > 2$ (Bishop 4.5)

1. The decision boundaries are very sensitive to outliers
 $\rightarrow$ easy points but the least squares loss wants for every point $y(x) \approx 1$ when $t_n = 1$
2. For $K > 2$ some decision regions can become very small or are even completely ignored
3. The components of $\mathbf{y}_{LS}(\mathbf{x})$ are not probabilities!

$y_k(\mathbf{x})$
 if $\sum_{k=1}^K t_k = 1$ $\xrightarrow{\text{then}}$ $\sum_k y_k = 1$

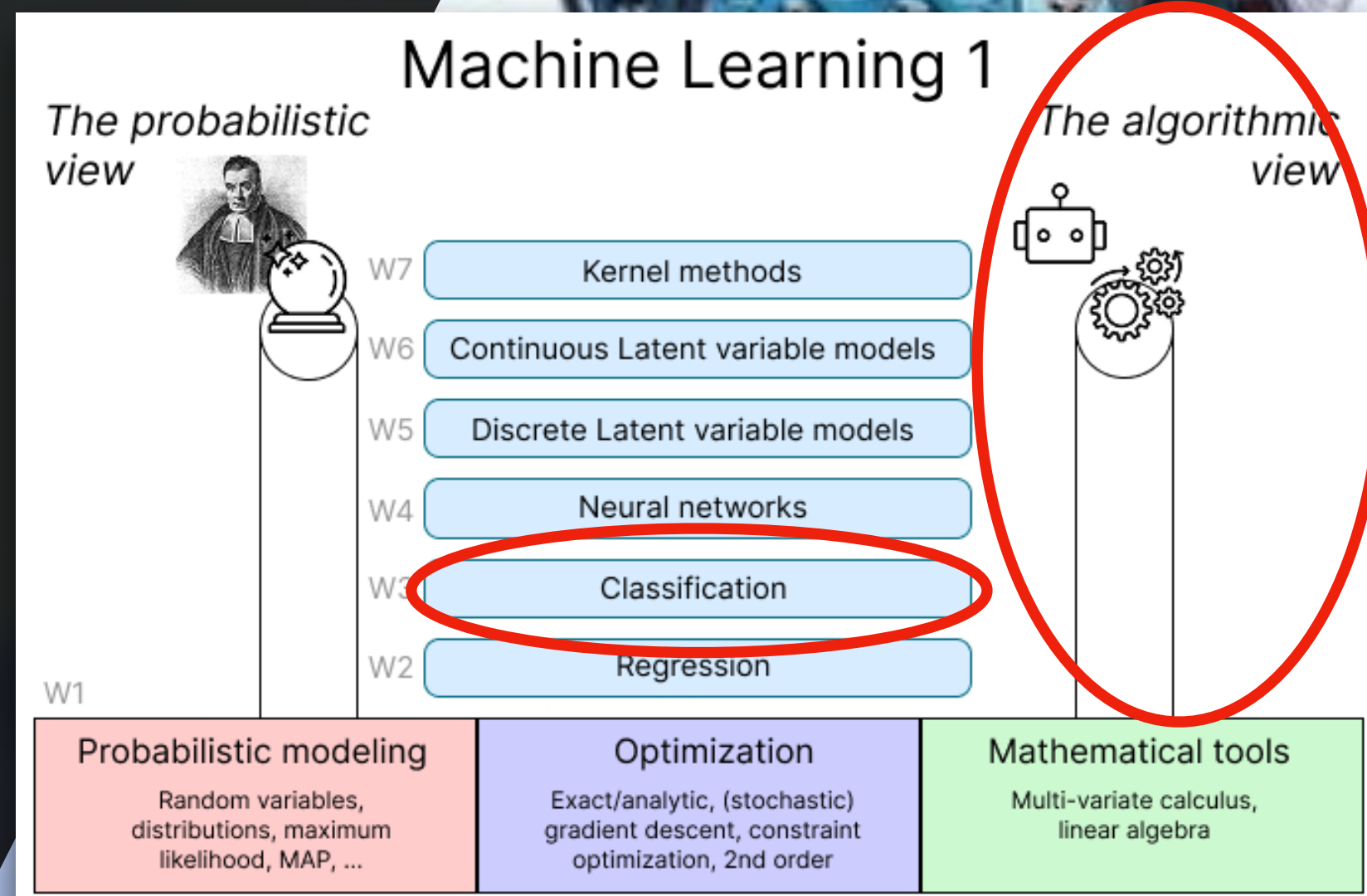


Machine Learning 1

Lecture 6.5 - Supervised Learning
Classification - Discriminative Models -
The Perceptron

Erik Bekkers

(Bishop 4.1.7)



The Perceptron Algorithm

Input: $\mathbf{x} \in \mathbb{R}^D$

Targets: $t \in \{C_1, C_2\} = \{1, -1\}$

Prediction: $y(\mathbf{x}) = f(\underbrace{\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})}_a)$, with $f(a) = \begin{cases} 1 & , a \geq 0 \\ -1 & , a < 0 \end{cases}$

Class decisions: assign $\mathbf{x}$ to class C_1 if $\mathbf{w}^T \boldsymbol{\phi}_n \geq 0$
(and to C_2 if $\mathbf{w}^T \boldsymbol{\phi} < 0$)

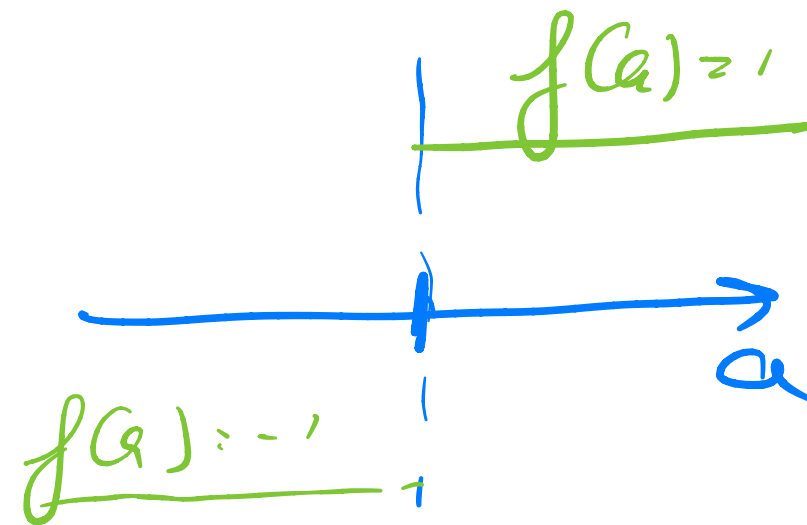
For correct classification: find $\mathbf{w} \neq \mathbf{0}$ such that for all $(\mathbf{x}_n, t_n)$:

$$\underline{\mathbf{w}^T \boldsymbol{\phi}_n} \cdot \underline{t_n} \geq 0$$

both positive: +
both negative: +
otherwise: -

Perceptron criterion: $E_P(\mathbf{w}) = - \sum_{n \in \mathcal{M}} \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) t_n$

with $\mathcal{M} = \{n : \mathbf{w}^T \boldsymbol{\phi}_n t_n < 0\} \leftarrow$ set of incorrectly classified points



Perceptron: Stochastic Gradient Descent

$$\begin{aligned}
 \triangleright E_P(\mathbf{w}) &= - \sum_{n \in \mathcal{M}} \mathbf{w}^T \phi(\mathbf{x}_n) t_n \\
 &= \sum_{n \in \mathcal{M}} E_n(\mathbf{w})
 \end{aligned}$$

$E_n(\mathbf{w}) = -\mathbf{w}^T \phi(\mathbf{x}_n) t_n$

- Stochastic Gradient Descent (SGD). For each misclassified $\mathbf{x}_n$:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w})$$

$$= \mathbf{w}^{(\tau)} + \eta \phi_n t_n$$

(after each update we recompute the set of incorrect predictions and randomly select one point)

- If $\mathbf{X}$ is linearly separable, then perceptron SGD will converge

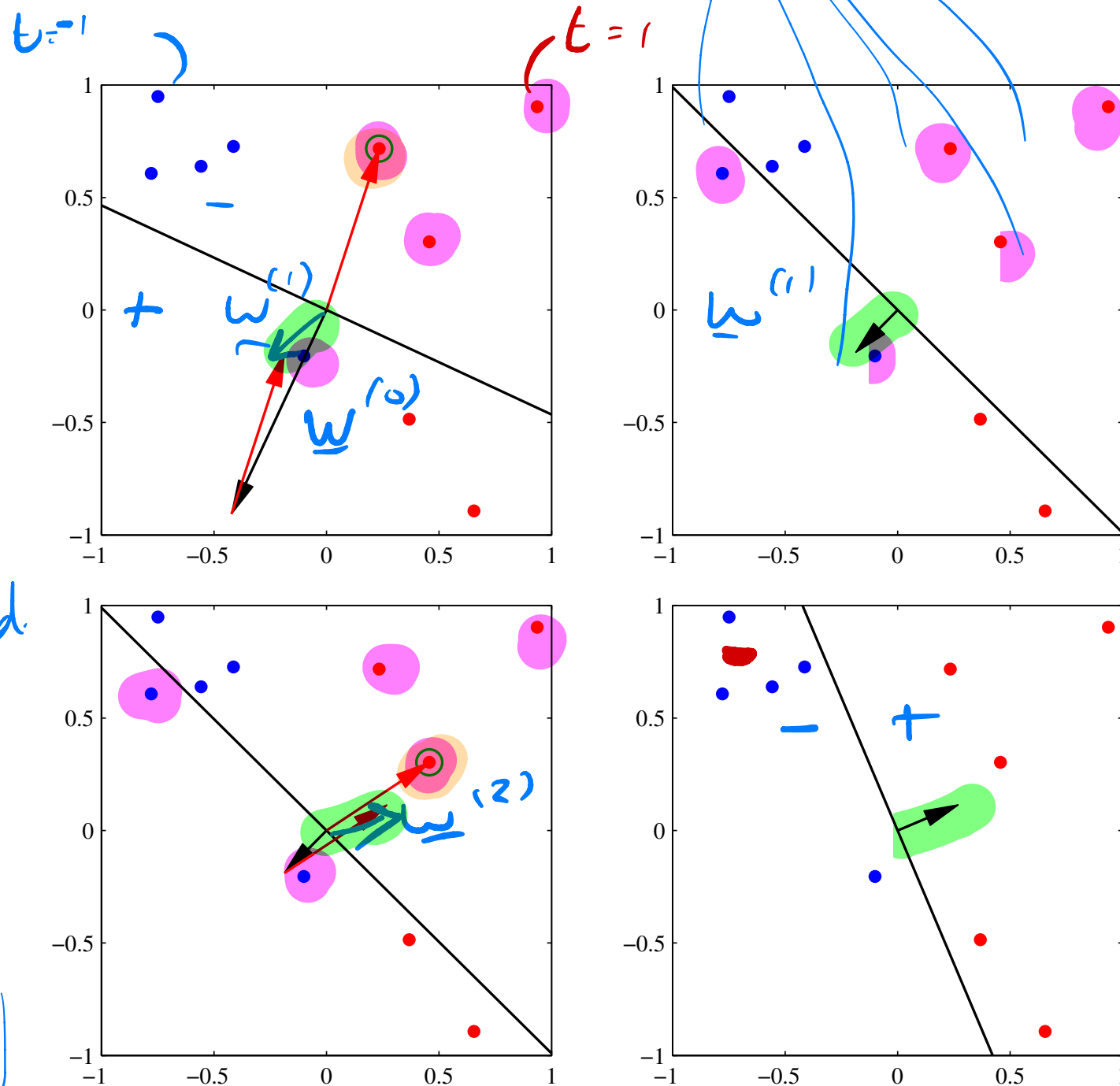


Figure: SGD for perceptron criterion (Bishop 4.7)
 for $\mathbf{x}_n$ in C_1 : add $\phi(\mathbf{x}_n)$ to $\mathbf{w}$
 for $\mathbf{x}_n$ in C_2 : subtract $\phi(\mathbf{x}_n)$ from $\mathbf{w}$.

Problems: Perceptron

- ▶ Perceptron only works for 2 classes
 - ▶ There might be many solutions depending on the initialization of $\mathbf{w}$ and on the order in which data is presented in SGD
 - ▶ If dataset is not linearly separable, the perceptron algorithm will not converge.
 - ▶ Based on linear combination of fixed basis functions.
- Random init*
- essentially determined by user*
- learning basis*
- via func multi-layer ~~perceptrons~~*

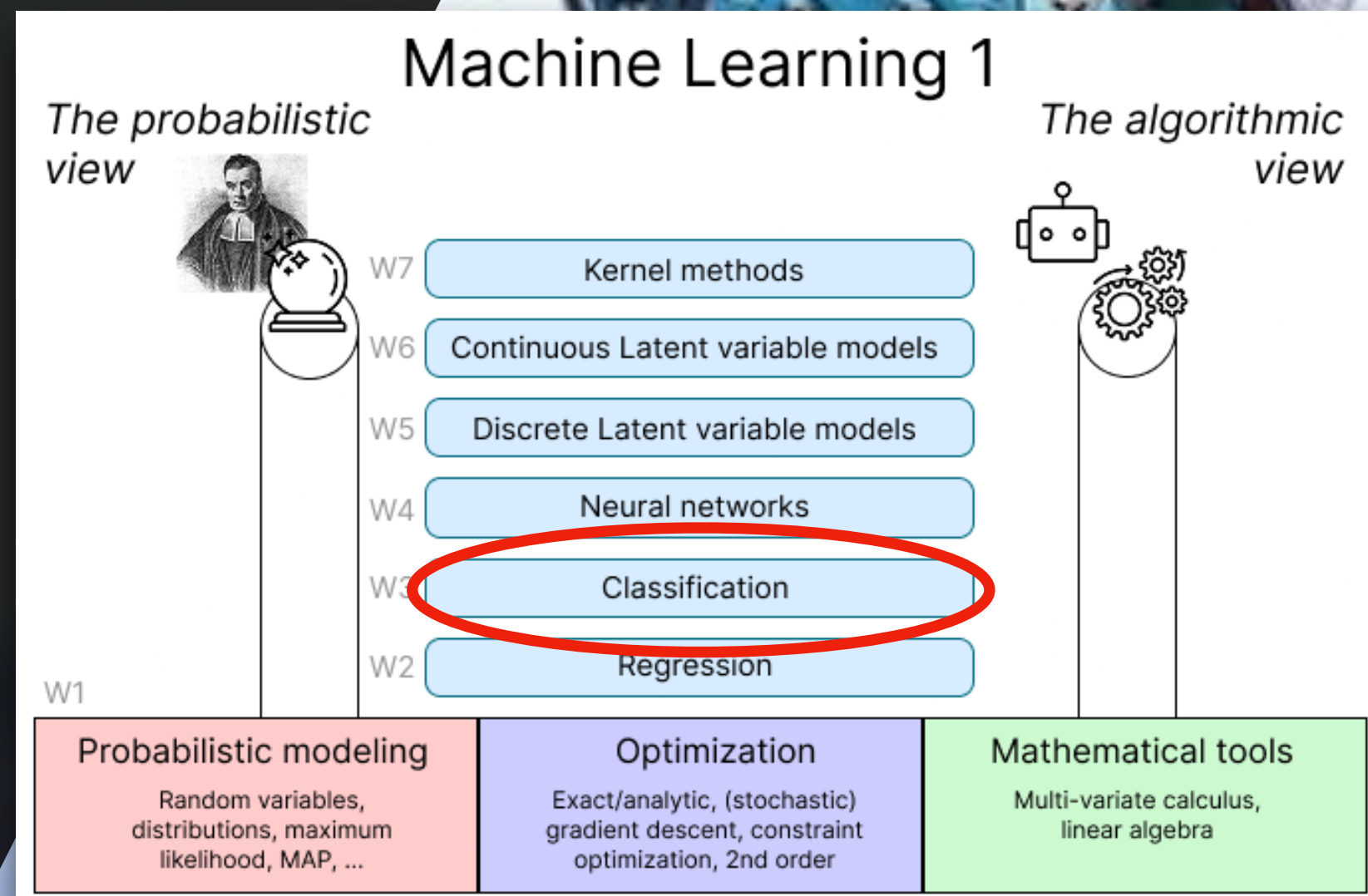


Machine Learning 1

Lecture 7.1 - Supervised Learning Classification - Classification With Basis Functions

Erik Bekkers

(Bishop 4.3.1)



Example: Use of Basis Functions

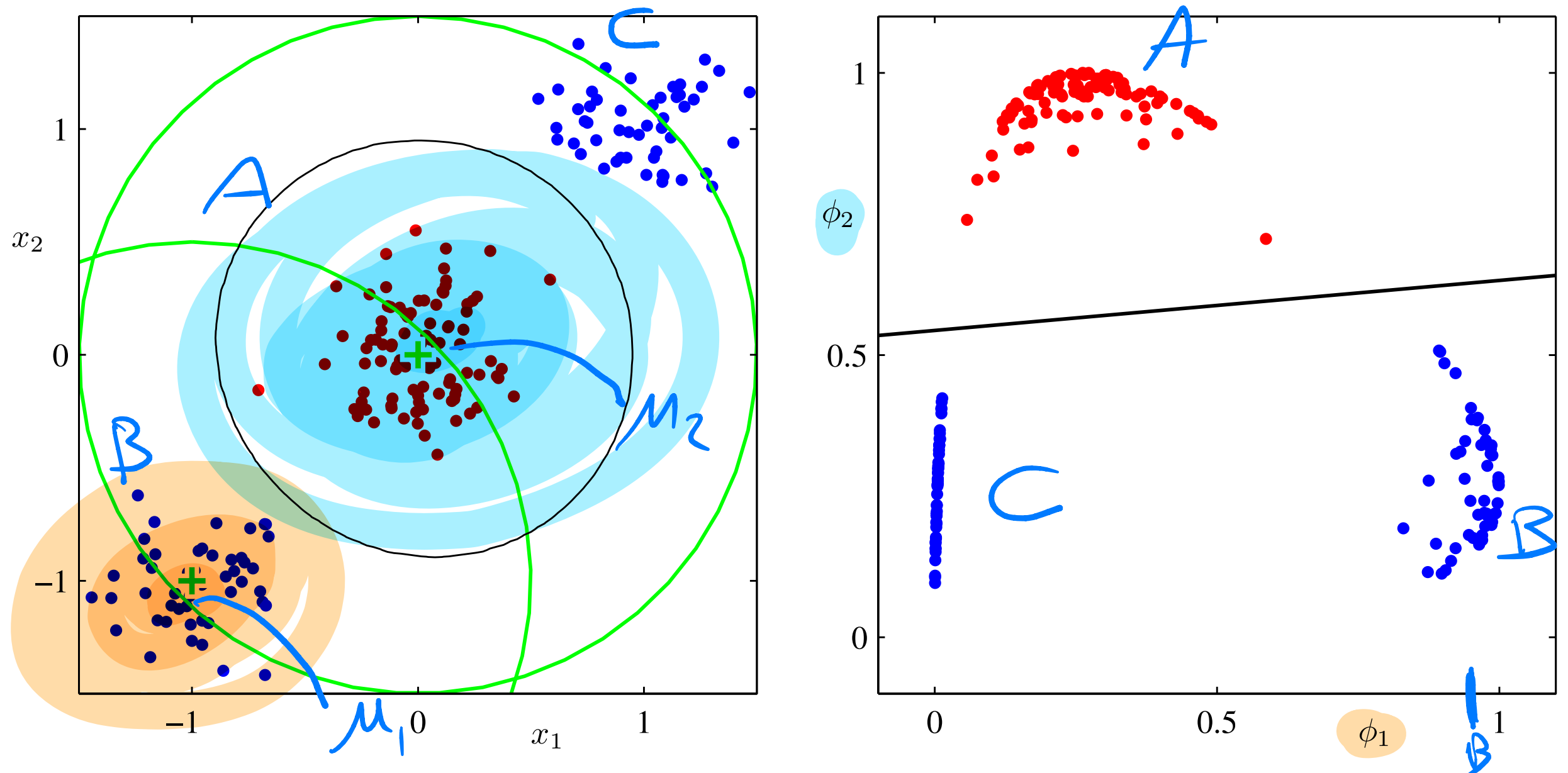


Figure: Left: original input space (x_1, x_2) , right: space of two gaussian basis functions with centres shown by the green crosses. (Bishop 4.12)

$$\phi_1(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_1)^T(\mathbf{x} - \mu_1)\right)$$

$$\phi_2(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \mu_2)^T(\mathbf{x} - \mu_2)\right)$$

I

Limitations of Fixed Basis Functions

Advantages:

- ✓ Closed form solution for least-squares problem
- (✓) ~~Tractable Bayesian treatment~~
- ✓ Nonlinear models mapping input variables to target variables through basis functions

neural networks

Limitations:

- ✗ Assumption: Basis functions $\phi_j(\mathbf{x})$ are fixed, not learned.
- ✗ Curse of dimensionality: to cover growing dimensions D of input vectors, the number of basis functions needs to grow rapidly / exponentially

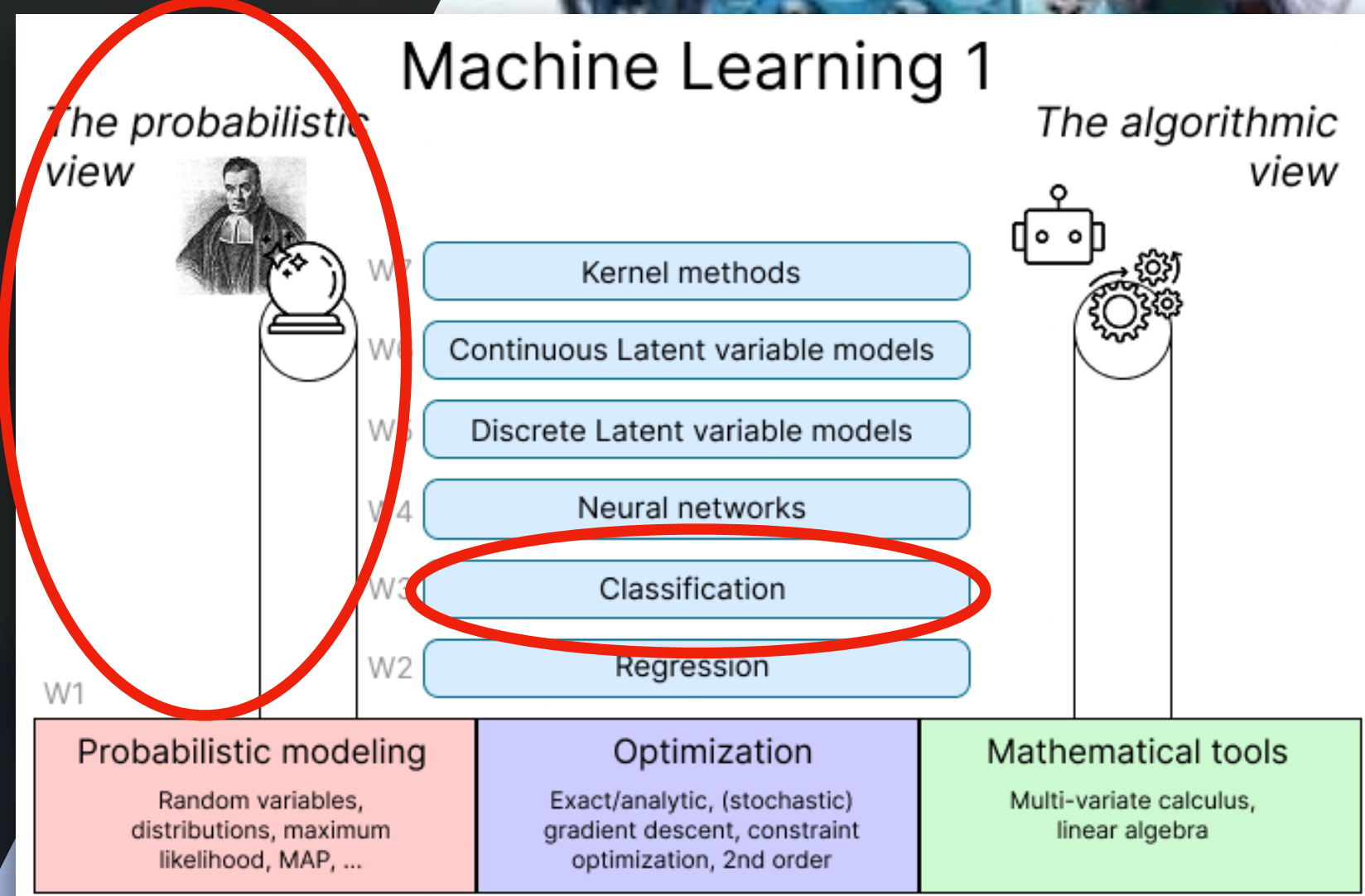


Machine Learning 1

Lecture 7.2 - Supervised Learning
Classification - Probabilistic Discriminative
Models - Logistic Regression

Erik Bekkers

(Bishop 4.3.2)



Classification Strategies

- Discriminant functions

Direct mapping of input to target:

e.g. perceptron
linear model
 $t = y(\mathbf{x}, \mathbf{w})$
 $\begin{cases} t=1 \\ t=0 \end{cases}$
 $y \geq 0$
 $y < 0$
 $(\mathbf{w}^T \mathbf{x})$

- Probabilistic discriminative models

See lecture 6 for why
it is natural to model
posteriors using logistic sigmoid

Posterior class probabilities:

$$p(C_k | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$$

- Probabilistic generative models

Class-conditional densities:

Prior class probabilities:

Gaussian:

LDA or QDA
 $p(\mathbf{x} | C_k)$
 $p(C_k)$
gives posterior of the form
 $p(C_k | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$
in terms of $\{\mu_k\}, \Sigma$

Logistic Regression for Two Classes

- Given: Dataset $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ with binary targets

$$\mathbf{t} = (t_1, \dots, t_N)^T \quad \text{with} \quad t_n \in \{C_1, C_2\} = \{1, 0\}$$

From now on we write

instead of 2

- Basis functions $\boldsymbol{\phi} := \boldsymbol{\phi}(\mathbf{x}) = (\phi_1(x), \phi_2(x), \dots)^T \in \mathbb{R}^M$

- Probabilistic Discriminative Linear Models:** Posteriors $p(C_k | \boldsymbol{\phi})$ are modeled with nonlinear functions f that take a linear function of $\boldsymbol{\phi}$ as input.

posteriors or probabilities in general are naturally expressed through the logistic sigmoid function

$$p(C_k | \boldsymbol{\phi}, \mathbf{w}) = f(\mathbf{w}^T \boldsymbol{\phi})$$

$$C_1 \leftrightarrow t=1, C_2 \leftrightarrow t=0$$

- Logistic regression (K=2)

$$p(C_1 | \boldsymbol{\phi}, \mathbf{w}) = y(\boldsymbol{\phi}) = \sigma(\mathbf{w}^T \boldsymbol{\phi})$$

$$p(C_2 | \boldsymbol{\phi}, \mathbf{w}) = 1 - y(\boldsymbol{\phi}) = 1 - \sigma(\mathbf{w}^T \boldsymbol{\phi})$$

$$p(t | \boldsymbol{\phi}, \mathbf{w}) = \underbrace{y(\boldsymbol{\phi})}_p(C_1 | \boldsymbol{\phi}, \mathbf{w})^t \underbrace{(1 - y(\boldsymbol{\phi}))}_{p(C_2 | \boldsymbol{\phi}, \mathbf{w})}^{1-t}$$

Combined into a single expression

Logistic Regression for Two Classes

- Let $\phi \in \mathbb{R}^M$ and M -dimensional feature vector
- Probabilistic Discriminative Modeling* with Logistic Regression:

$$p(C_1 | \phi, \mathbf{w}) = \sigma(\mathbf{w}^T \phi) \quad \text{and} \quad p(C_2 | \phi, \mathbf{w}) = 1 - \sigma(\mathbf{w}^T \phi)$$

parameters: $\underline{w}$; M parameters $\rightarrow$ Scales linearly

- Generative Modeling* with Gaussian conditional densities:

$$p(\mathbf{x} | C_k) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma_k|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k)\right\}$$

and class priors $p(C_k)$ + K params

parameters:

Scales quadratically with M

$\left\{ \begin{array}{l} \underline{\mu}_k : 2M \text{ parameters} \\ \underline{\Sigma} : \sim M^2 \text{ parameters} \end{array} \right.$

param could be further reduced with Naïve Bayes

Logistic Regression for Two Classes

- Given: Dataset $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ with binary targets

$$\mathbf{t} = (t_1, \dots, t_N)^T \quad \text{with} \quad t_n \in \{C_1, C_2\} = \{0, 1\}$$

- Conditional likelihood function:

$$p(\mathbf{t} | \mathbf{X}, \mathbf{w}) = \prod_{n=1}^N p(t_n | \mathbf{x}_n, \mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n}$$

$$p(x, t | y)$$

$$y_n = p(C_1 | \phi_n) = \sigma(\mathbf{w}^T \phi_n)$$

$$\phi_n := \phi(\mathbf{x}_n)$$

- Maximizing the conditional likelihood/minimizing the cross-entropy

$$E(\mathbf{w}) = -\ln p(\mathbf{t} | \mathbf{X}, \mathbf{w}) = -\sum_{n=1}^N \{t_n \ln y_n + (1-t_n) \ln (1-y_n)\}$$

- $E(\mathbf{w})$: convex, but no closed form solution!

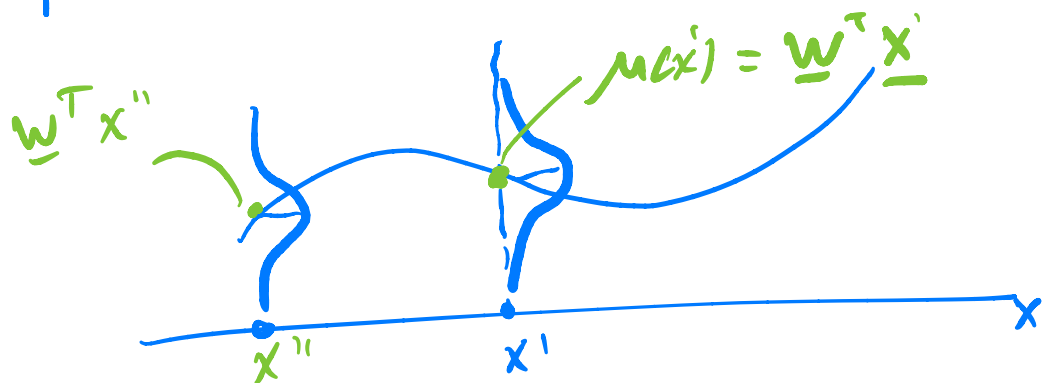
$$y_n = \sigma(\mathbf{w}^T \phi_n) \text{ is nonlinear in } \mathbf{w}$$

$$H_{ij} = \frac{\partial^2}{\partial w_i \partial w_j} E(\mathbf{w})$$

see lecture 7.4.
(end of these slides) $\Rightarrow \forall x: x^T H x \geq 0$

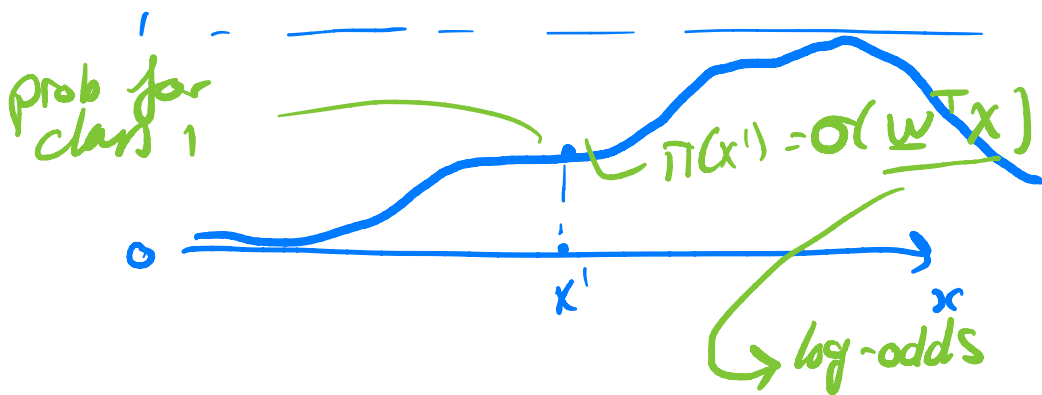
Regression w Gaussian predictive

$$p(t' | \underline{x}', \underline{w}) = \mathcal{N}(t' | \mu(\underline{x}'), \sigma^2)$$



Classification w Bernoulli pred. distr.

$$\begin{aligned} p(t' | \underline{x}', \underline{w}) &= \text{Bernoulli}(t' | \pi(\underline{x}')) \\ &= \pi(\underline{x}')^{t'} (1 - \pi(\underline{x}'))^{1-t'} \end{aligned}$$



The cross-entropy loss

$$E_n(\mathbf{w}) = - \{ \underbrace{t_n \ln(y_n)}_{\text{huge LS loss}} + (1 - t_n) \ln(1 - y_n) \}$$

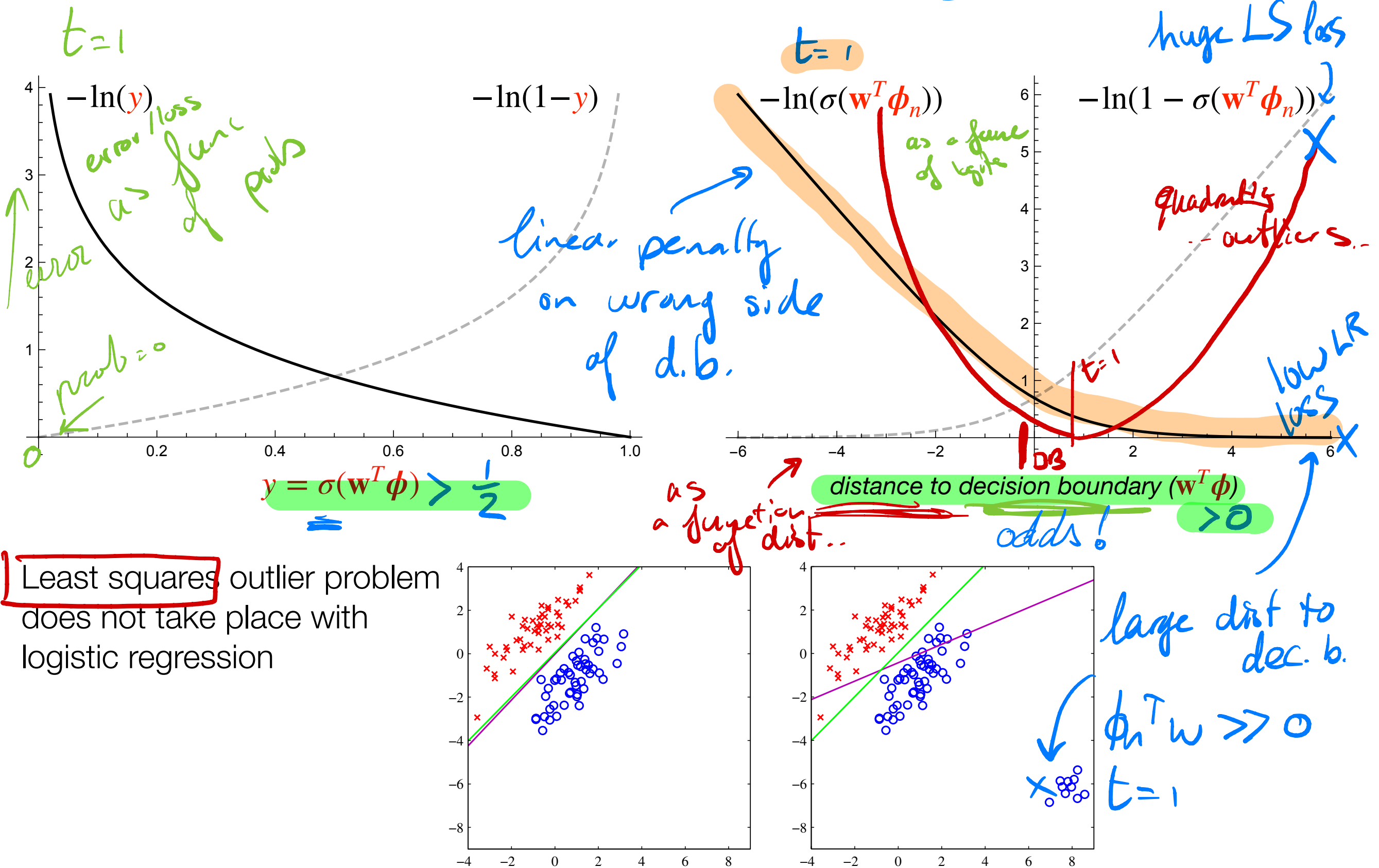


Figure: least squares is very sensitive to outliers (Bishop 4.4)

Classification with Logistic Regression

- Given: Dataset $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ with binary targets

$$\mathbf{t} = (t_1, \dots, t_N)^T \quad \text{with} \quad t_n \in \{C_1, C_2\} = \{0, 1\}$$

- Basis functions: $\boldsymbol{\phi}(\mathbf{x}) = (\phi_0(\mathbf{x}), \dots, \phi_{M-1}(\mathbf{x}))^T$

- Posterior distributions (Prob. Discr. Model): $p(C_1 | \mathbf{x}, \mathbf{w}) = \sigma(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}))$

- Minimize

$$E(\mathbf{w}) = -\ln p(\mathbf{t} | \mathbf{X}, \mathbf{w}) = -\sum_{n=1}^N t_n \ln y_n + (1 - t_n) \ln(1 - y_n)$$

Convex
so
unique

with *stochastic gradient descent* or *iterative reweighted least squares*, to find $\mathbf{w}^*$

- New datapoint $\mathbf{x}'$ is assigned to class C_1 if

$$p(C_1 | \mathbf{x}', \mathbf{w}^*) = \sigma((\mathbf{w}^*)^T \boldsymbol{\phi}(\mathbf{x}')) > \frac{1}{2}$$

$$(\mathbf{w}^*)^T \boldsymbol{\phi} > 0$$

- Decision boundaries:

linear

$$\mathbf{w}^*^T \boldsymbol{\phi} = 0$$

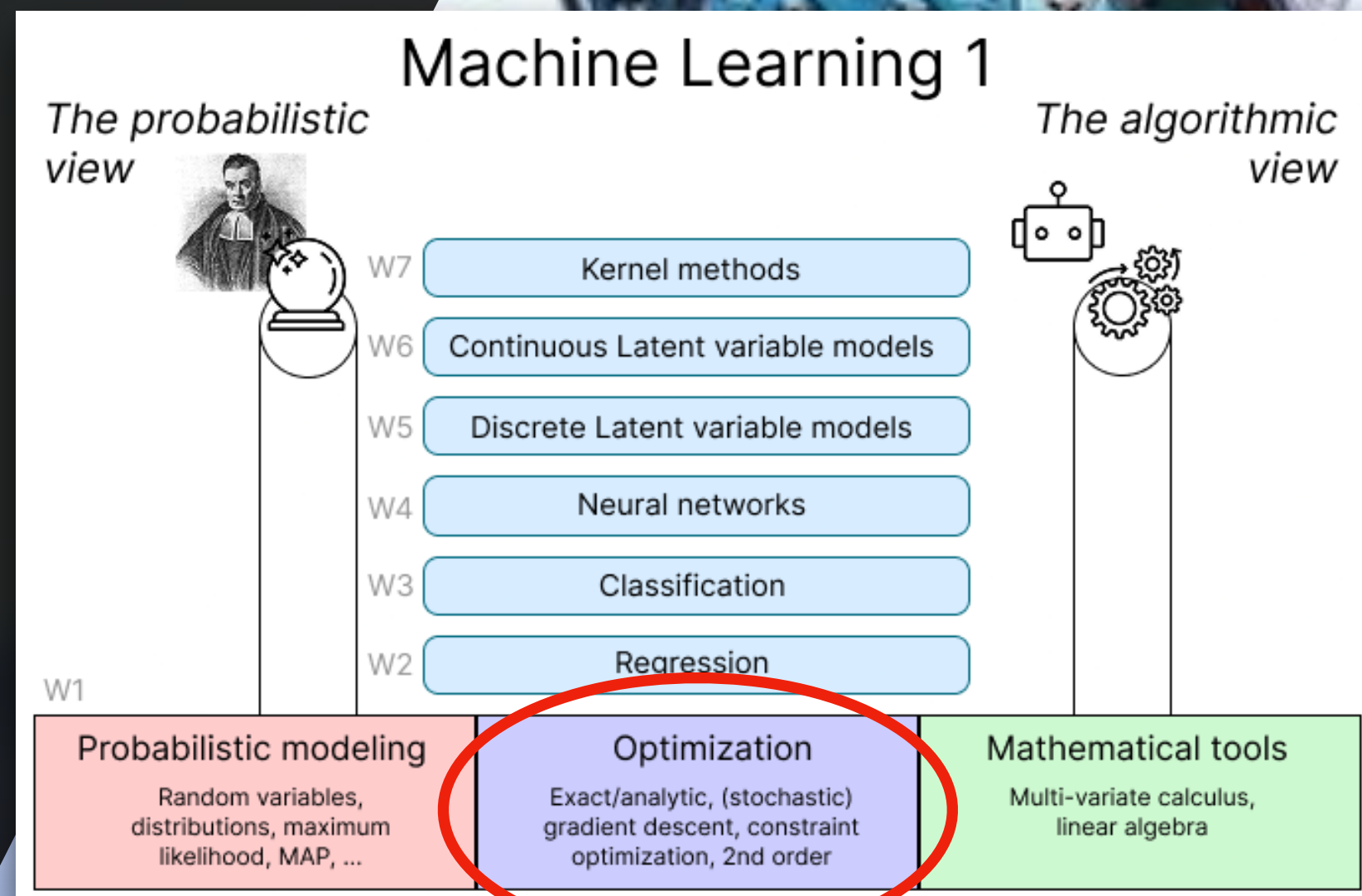


Machine Learning 1

Lecture 7.3 - Supervised Learning
Classification - Logistic Regression:
Stochastic Gradient Descent

Erik Bekkers

(Bishop 4.3.2)



Has very simple gradient! Logistic Regression (K=2): SGD

- Stochastic Gradient Descent for cross-entropy:

$$E(\mathbf{w}) = - \sum_{n=1}^N t_n \ln y_n + (1 - t_n) \ln(1 - y_n) = \sum_{n=1}^N E_n(\mathbf{w})$$

$\hookrightarrow y_n = \sigma(\mathbf{w}^T \phi)$

- Update rule given a random data point $(\mathbf{x}_n, t_n)$

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

- Gradient: $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right)^T$

$$\frac{\partial E_n(\mathbf{w})}{\partial w_j} = \frac{\partial E_n(\mathbf{w})}{\partial y_n} \cdot \frac{\partial y_n}{\partial w_j} = \left(-\frac{t_n}{y_n} + \frac{1-t_n}{1-y_n} \right) \cdot \frac{\partial y_n}{\partial w_j}$$

$$\frac{\partial y_n}{\partial w_j} = \frac{\partial}{\partial w_j} \sigma(\mathbf{w}^T \phi_n)$$

$\leftarrow$

Logistic Regression (K=2): SGD

Let's derive $\frac{\partial y_n}{\partial w_j} = \frac{\partial}{\partial w_j} \sigma(\mathbf{w}^T \boldsymbol{\phi}_n)$ using $\frac{\partial \sigma(a)}{\partial a} = \sigma(a)(1 - \sigma(a))$

$$\frac{\partial}{\partial w_j} \sigma(\mathbf{w}^T \boldsymbol{\phi}_n) = \sigma(\mathbf{w}^T \boldsymbol{\phi}_n)(1 - \sigma(\mathbf{w}^T \boldsymbol{\phi}_n)) \frac{\partial (\mathbf{w}^T \boldsymbol{\phi}_n)}{\partial w_j}$$

Handwritten note: $\frac{\partial}{\partial w_j} (\sum_m w_m \phi_{nm})$
 $m=j: \frac{\partial}{\partial w_j} \neq 0$
 $= \phi_{nj}$

$$= y_n (1 - y_n) \phi_{nj}$$

$\frac{\partial E_n(\mathbf{w})}{\partial w_j} = -\frac{t_n}{y_n} \frac{\partial y_n}{\partial w_j} + \frac{1 - t_n}{1 - y_n} \frac{\partial y_n}{\partial w_j}$

$$= -\frac{t_n}{y_n} y_n (1 - y_n) \phi_{nj} + \frac{1 - t_n}{1 - y_n} y_n (1 - y_n) \phi_{nj}$$

$$= -t_n \phi_{nj} + \cancel{t_n y_n \phi_{nj}} + \phi_{nj} - \cancel{t_n y_n \phi_{nj}}$$

$$= (y_n - t_n) \phi_{nj}$$

Logistic Regression (K=2): SGD

- Stochastic Gradient Descent for cross-entropy:

$$E(\mathbf{w}) = - \sum_{n=1}^N t_n \ln y_n + (1 - t_n) \ln(1 - y_n)$$

- Update rule given a random data point $(\mathbf{x}_n, t_n)$

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla E_n(\mathbf{w}^{(\tau)})$$

- Gradient: $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right)^T = (y_n - t_n) \boldsymbol{\phi}_n$

Simple form b/c activation function f is the **link function**! (Bishop 4.3.6)

error $\times$ feature vector

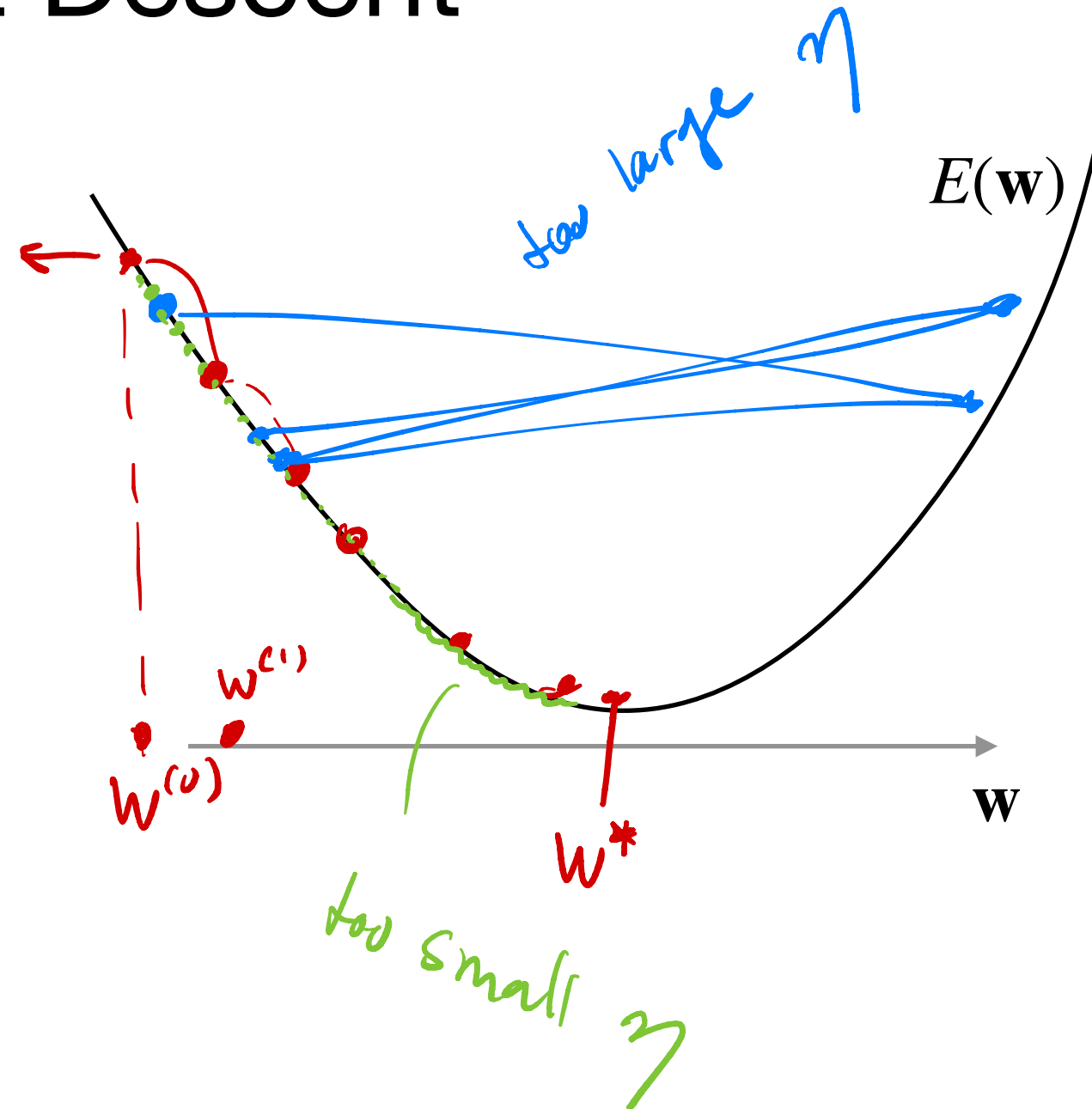
- Gradient Descent for LR has a perceptron-like update rule:

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta (y_n - t_n) \boldsymbol{\phi}_n$$

Handwritten blue scribbles

Stochastic Gradient Descent

1. Initialize $\mathbf{w}^{(0)}$
2. Choose a learning rate η
3. While $\|\mathbf{w}^{(\tau+1)} - \mathbf{w}^{(\tau)}\| > \epsilon$
 - I. Choose a random data point $(\mathbf{x}_n, t_n)$
 - II. Update $\mathbf{w}$:
$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta(y_n - t_n)\phi_n$$



- ▶ If η too large: no convergence
- ▶ If η too small: very slow convergence
- ▶ Converged $\mathbf{w}^*$: estimate of minimizer of $E(\mathbf{w})$!

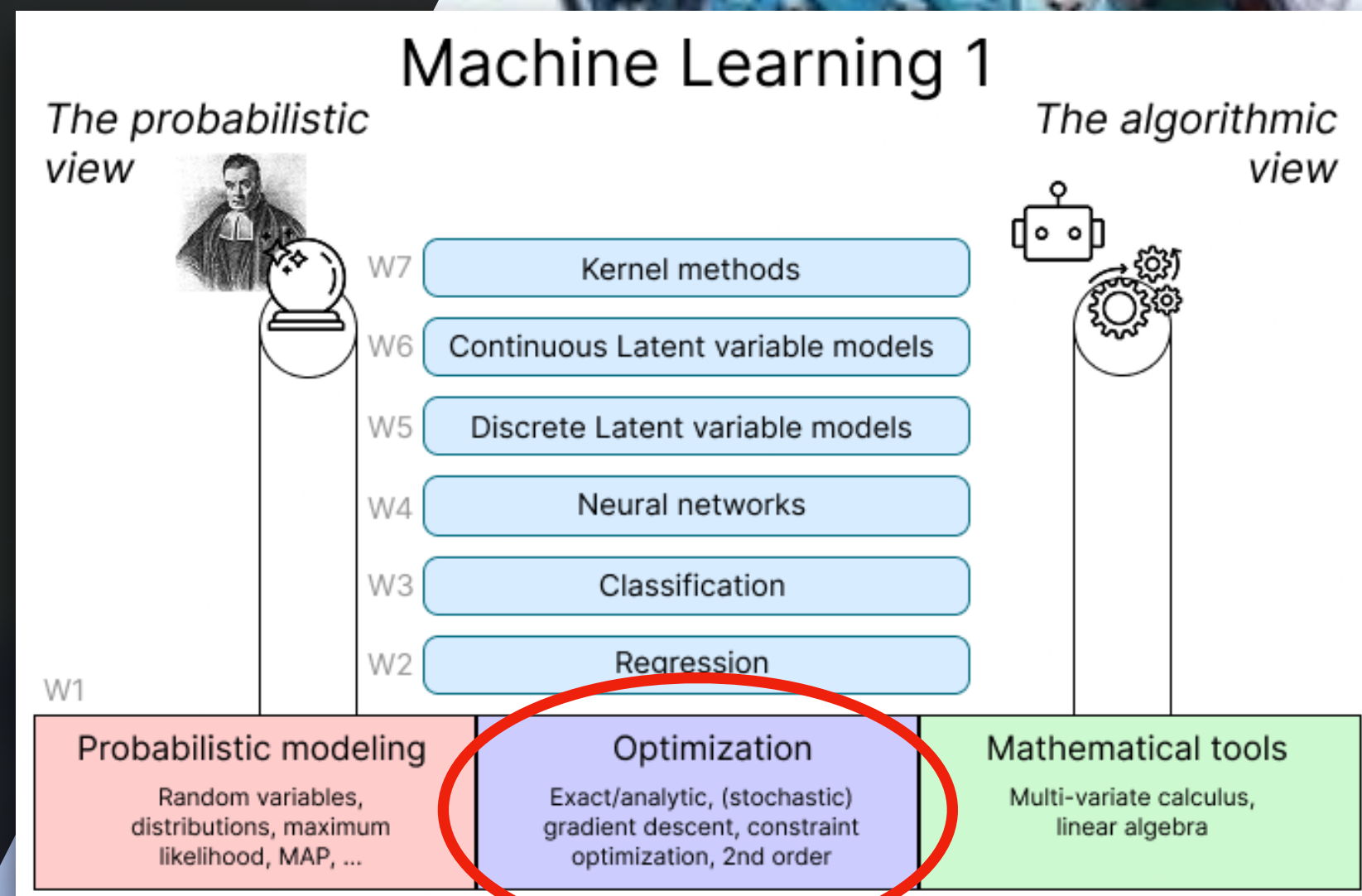


Machine Learning 1

Lecture 7.4 - Supervised Learning
Classification - Logistic Regression: Newton-
Raphson Optimization

Erik Bekkers

(Bishop 4.3.3)



Newton-Raphson Iterative Optimization

- Goal: minimize

$$E(\mathbf{w}) = - \sum_{n=1}^N t_n \ln y_n + (1 - t_n) \ln(1 - y_n)$$

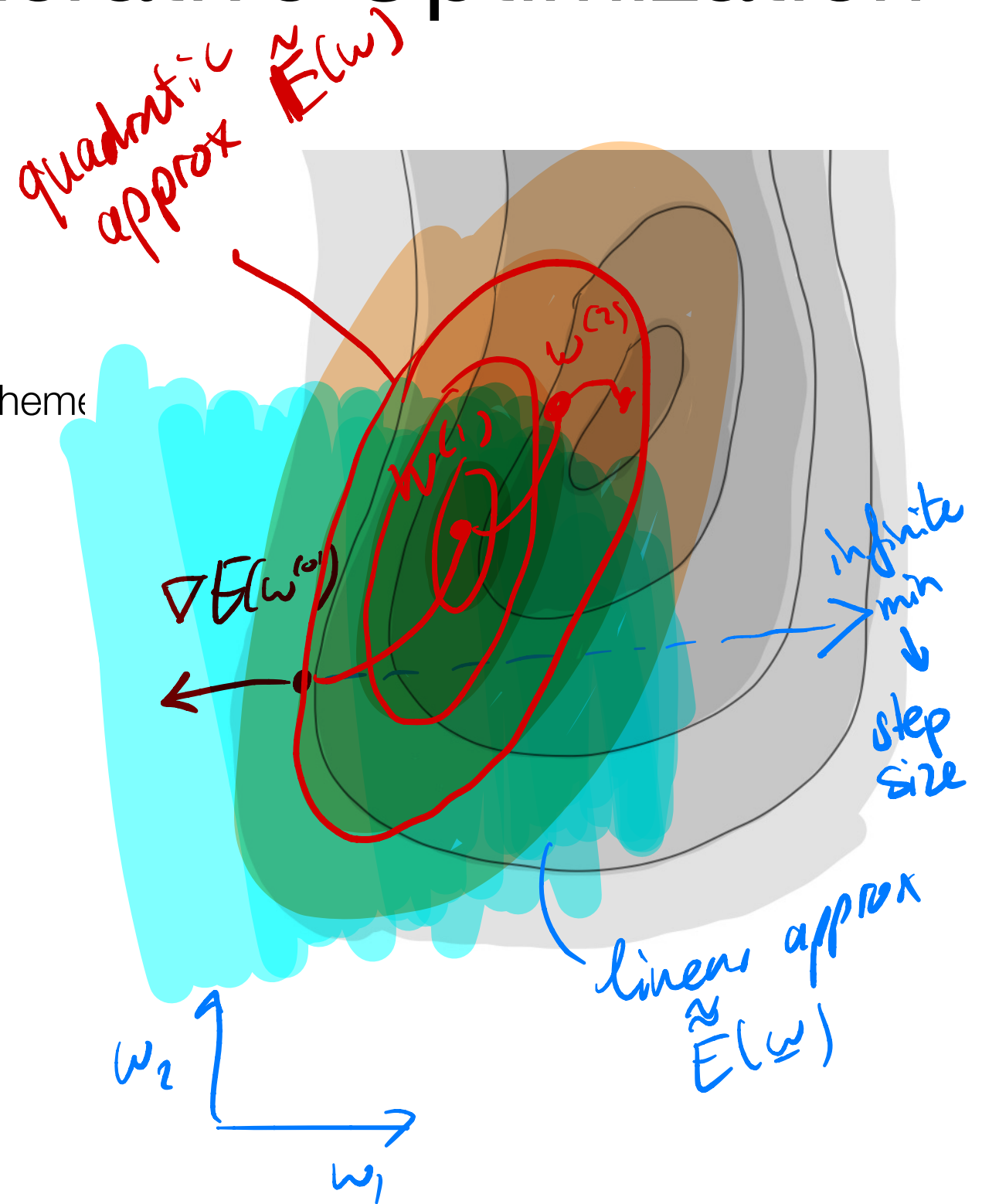
- Newton-Raphson iterative second order optimization scheme

- Initial guess $\mathbf{w}^0$
- For $\tau = 1, \dots$:
 - Approximate $E(\mathbf{w})$ with a quadratic function $\tilde{E}_{\tau-1}(\mathbf{w})$ around $\mathbf{w}^{(\tau-1)}$
 - Set $\mathbf{w}^{(\tau)}$ such that it minimizes $\tilde{E}(\mathbf{w})$, i.e.,

$$\mathbf{w}^{(\tau)} = \underset{\mathbf{w}}{\operatorname{argmin}} \tilde{E}_{\tau-1}(\mathbf{w})$$
 - Stop when $\|\mathbf{w}^{(\tau+1)} - \mathbf{w}^{(\tau)}\| = 0$

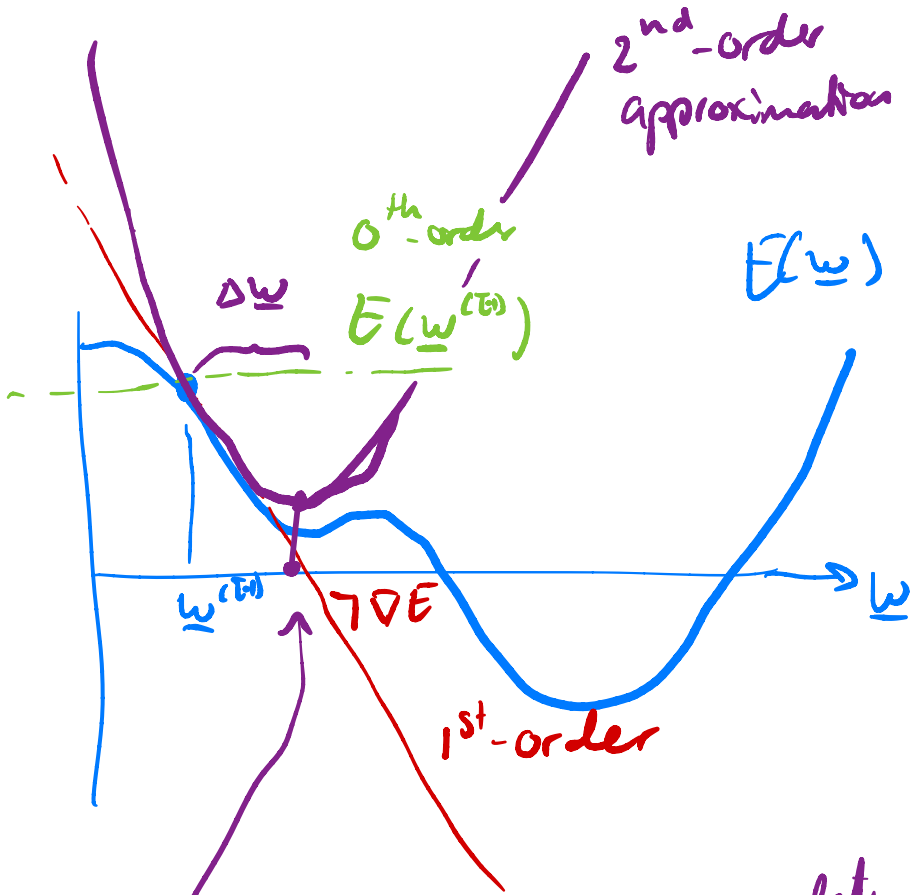
So no
step size

- You have found $\mathbf{w}^*$ such that $\frac{\partial}{\partial \mathbf{w}} E(\mathbf{w}^*) = 0$



Taylor expansion:

$$f(x_0 + \Delta x) \approx \underbrace{f(x_0)}_{\text{0th order}} + \underbrace{\Delta x \frac{d}{dx} f|_{x_0}}_{\text{1st order}} + \underbrace{\frac{1}{2} \Delta x^2 \frac{d^2}{dx^2} f|_{x_0}}_{\text{2nd order}} + \dots \text{higher order derivatives}$$



well-defined minimum so let

$$w^{(ct)} = \underset{w}{\operatorname{argmin}} E_{w^{(ct)}}^{(2)}(w)$$

$$= w^{(ct-1)} + \Delta w$$

Newton-Raphson Iterative Optimization

- Given your old estimate $\mathbf{w}^{(\tau-1)}$, approximate $E(\mathbf{w})$ with a second order Taylor expansion around $\mathbf{w}^{(\tau-1)}$

$$E(\mathbf{w}) \approx \tilde{E}(\mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}) = E(\mathbf{w}^{(\tau-1)}) + \underbrace{(\Delta\mathbf{w})^T \nabla^T E(\mathbf{w}^{(\tau-1)})}_{\text{1st-order}} + \frac{1}{2} \underbrace{(\Delta\mathbf{w})^T \mathbf{H} \Delta\mathbf{w}}_{\text{2nd-order}}$$

0th-order
1st-order
2nd-order

- Gradient (first order derivatives):

$$\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right)$$

- Hessian Matrix (second order derivatives):

$$H_{ij} = \frac{\partial^2 E(\mathbf{w})}{\partial w_i \partial w_j}$$

- Choose $\Delta\mathbf{w}$ such that next estimate $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}$ minimizes $\tilde{E}(\mathbf{w})$:

$$\frac{\partial}{\partial \Delta\mathbf{w}} \tilde{E}(\mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}) = \nabla E(\mathbf{w}^{(\tau-1)}) + (\Delta\mathbf{w})^T \mathbf{H} = 0 \rightarrow \mathbf{H} \Delta\mathbf{w} = -\nabla^T E$$

$$\rightarrow \Delta\mathbf{w} = -\mathbf{H}^{-1} \nabla^T E$$

- Update rule:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$$

optimal step size!

Newton-Raphson Iterative Optimization

▸ Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

▸ Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \boldsymbol{\phi}_n$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}_n =$$

$$\left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

Newton-Raphson Iterative Optimization

Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \phi_n$

all data

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n = \underbrace{\Phi^T}_{\text{feat vec}} \underbrace{(\mathbf{y} - \mathbf{t})}_{\text{error}}$$

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j}$

Newton-Raphson Iterative Optimization

Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \phi_n^T$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n = \Phi^T (\mathbf{y} - \mathbf{t})$$

$$\begin{pmatrix} \phi_1(\mathbf{x}_n) \\ \phi_2(\mathbf{x}_n) \\ \vdots \end{pmatrix} \quad \left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j} = \frac{\partial}{\partial w_i} \sum_{n=1}^N (y_n - t_n) \phi_j(\mathbf{x}_n)$

$\underline{\mathbf{w}}^T \underline{\phi}_n$

Newton-Raphson Iterative Optimization

▸ Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

▸ Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \boldsymbol{\phi}_n$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}_n = \boldsymbol{\Phi}^T (\mathbf{y} - \mathbf{t})$$

$$\left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

▸ Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j} = \frac{\partial}{\partial w_i} \sum_{n=1}^N (y_n - t_n) \phi_j(\mathbf{x}_n) = \sum_{n=1}^N \phi_j(\mathbf{x}_n) \frac{\partial y_n}{\partial w_i}$

Newton-Raphson Iterative Optimization

Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \boldsymbol{\phi}_n$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}_n = \boldsymbol{\Phi}^T (\mathbf{y} - \mathbf{t})$$

$$\left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j} = \frac{\partial}{\partial w_i} \sum_{n=1}^N (y_n - t_n) \phi_j(\mathbf{x}_n) = \sum_{n=1}^N \phi_j(\mathbf{x}_n) \frac{\partial y_n}{\partial w_i}$

$$y_n = \sigma(\boldsymbol{\phi}_n^T \mathbf{w})$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$= \sum_{n=1}^N y_n(1 - y_n) \phi_i \phi_j$$

"weight per data point"

Newton-Raphson Iterative Optimization

Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \boldsymbol{\phi}_n$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}_n = \boldsymbol{\Phi}^T (\mathbf{y} - \mathbf{t})$$

$$\left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j} = \frac{\partial}{\partial w_i} \sum_{n=1}^N (y_n - t_n) \phi_j(\mathbf{x}_n) = \sum_{n=1}^N \phi_j(\mathbf{x}_n) \frac{\partial y_n}{\partial w_i}$

$\in \mathbb{R}^{M \times M}$

$R = \text{diag}(\{y_n(1-y_n)\})$

$$= \sum_{n=1}^N y_n(1 - y_n) \phi_i \phi_j$$

$$\mathbf{H} = \sum_{n=1}^N y_n(1 - y_n) \boldsymbol{\phi}_n \boldsymbol{\phi}_n^T =$$

$$\boldsymbol{\Phi}^T \mathbf{R} \boldsymbol{\Phi}$$

$\rightarrow$

$$\begin{cases} R_{nn} = y_n(1 - y_n) \\ R_{nm} = 0 \text{ if } n \neq m \end{cases}$$

Newton-Raphson Iterative Optimization

Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$

Gradient $\nabla E_n(\mathbf{w}) = \left(\frac{\partial E_n(\mathbf{w})}{\partial w_0}, \dots, \frac{\partial E_n(\mathbf{w})}{\partial w_{M-1}} \right) = (y_n - t_n) \boldsymbol{\phi}_n$

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}_n = \boldsymbol{\Phi}^T (\mathbf{y} - \mathbf{t})$$

$$\left(E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w}) \right)$$

Hessian $H_{ij} = \frac{\partial^2 E(\mathbf{w}^{(\tau-1)})}{\partial w_i \partial w_j} = \frac{\partial}{\partial w_i} \sum_{n=1}^N (y_n - t_n) \phi_j(\mathbf{x}_n) = \sum_{n=1}^N \phi_j(\mathbf{x}_n) \frac{\partial y_n}{\partial w_i}$

$$= \sum_{n=1}^N y_n (1 - y_n) \phi_i \phi_j$$

$$\mathbf{H} = \sum_{n=1}^N y_n (1 - y_n) \boldsymbol{\phi}_n \boldsymbol{\phi}_n^T = \boldsymbol{\Phi}^T \mathbf{R} \boldsymbol{\Phi}$$

$$\Delta \mathbf{w} = -\mathbf{H}^{-1} \nabla E^T$$

Iterative Reweighted Least Squares

- Update rule: $\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \mathbf{H}^{-1} \nabla^T E(\mathbf{w}^{(\tau-1)})$ *optimal Δw give the approx.*
- With Gradient: $\nabla E(\mathbf{w}) = \Phi^T (\mathbf{y} - \mathbf{t})$
- With Hessian: $\mathbf{H} = \Phi^T \mathbf{R} \Phi$, with $\mathbf{R} = \text{diag}_{N \times N} \{y_n(1 - y_n)\}$
- Newton-Raphson updates solutions to a weighted least squares problem $\rightarrow$ iterative reweighted least squares:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T (\mathbf{y} - \mathbf{t})$$

$$= (\Phi^T \mathbf{R} \Phi)^{-1} \{ (\Phi^T \mathbf{R} \Phi) \mathbf{w}^{(\tau-1)} - \Phi^T (\mathbf{y} - \mathbf{t}) \}$$

$$= (\Phi^T \mathbf{R} \Phi)^{-1} \{ \Phi^T \mathbf{R} (\mathbf{z} + \mathbf{R}^{-1} (\mathbf{y} - \mathbf{t})) - \Phi^T (\mathbf{y} - \mathbf{t}) \}$$

$$= (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T \mathbf{R} \mathbf{z}$$

with $\mathbf{z} = \Phi \mathbf{w}^{(\tau-1)} - \mathbf{R}^{-1} (\mathbf{y} - \mathbf{t})$

Note similarity with ML solution to linear regression:

$$\mathbf{w}_{ML} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$$

(solution to least squares regression $\mathbf{w}_{ML} = \underset{\mathbf{w}}{\text{argmin}} \sum_{n=1}^N (\mathbf{w}^T \phi_n - t_n)^2$)

$$\mathbf{w}^{(\tau)} = (\Phi^T \mathbf{R} \Phi)^{-1} \Phi^T \mathbf{R} \mathbf{z}$$

(solution to a weighted least squares problem $\mathbf{w}^{(\tau)} = \underset{\mathbf{w}}{\text{argmin}} \sum_{n=1}^N r_i (\mathbf{w}^T \phi_n - z_n)^2$)

SGD vs Newton-Raphson

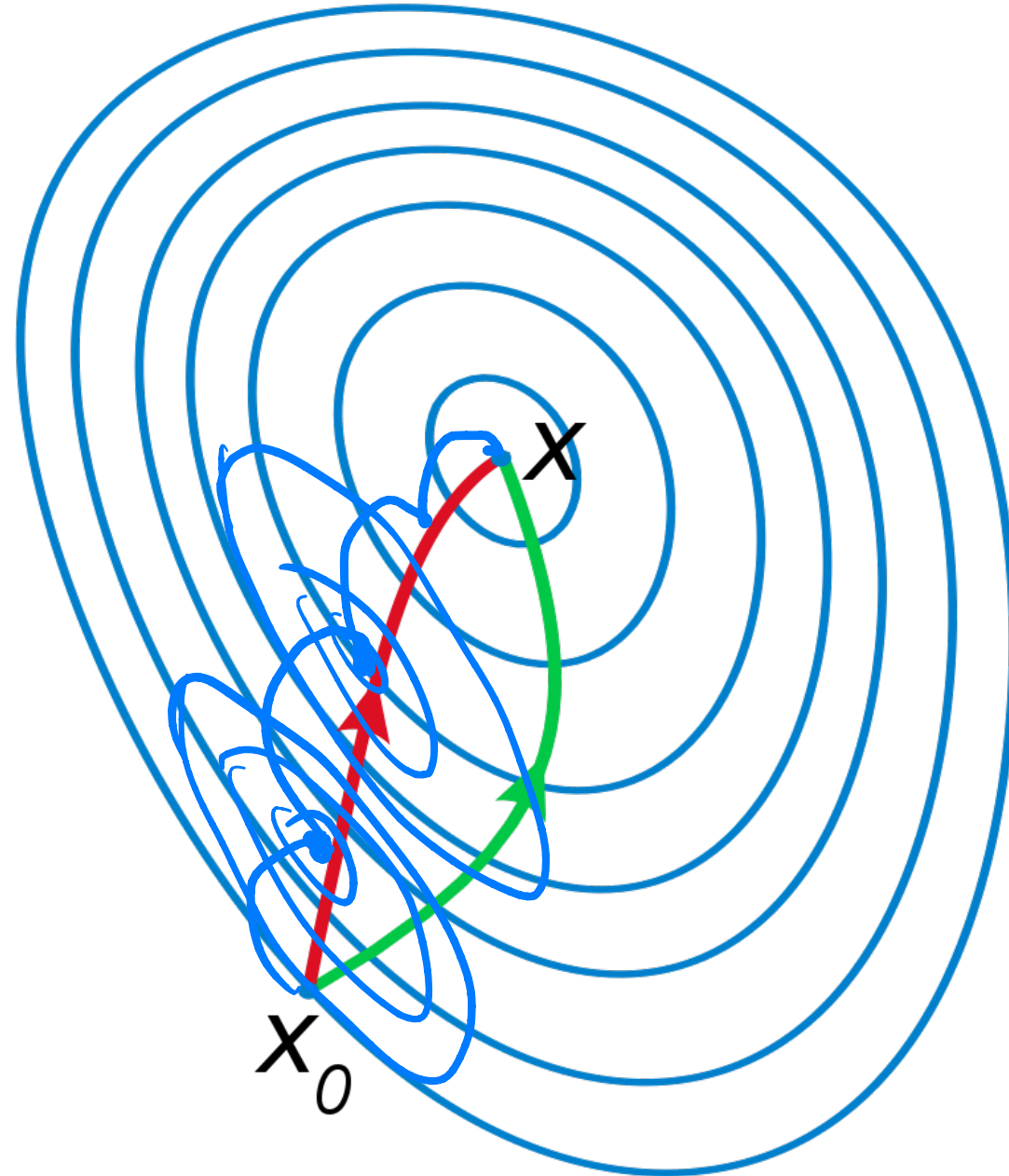


Figure: Green: gradient descent, which always goes in the direction of steepest descent. Red: Newton-Raphson's procedure, which takes into account curvature to take a more direct path. (Wikipedia)

The cross entropy loss is convex

- ▶ The error function $E(\mathbf{w})$ is convex since the Hessian is ^{semi-}positive definite
- ▶ *Proof Hessian is convex:*

The Hessian of $E(\mathbf{w})$ is given by $\mathbf{H} = \Phi^T \mathbf{R} \Phi$, with $\mathbf{R} = \text{diag}_{N \times N}\{y_n(1 - y_n)\}$.

To show: $\forall \mathbf{w} \neq \mathbf{0} \in \mathbb{R}^M : \mathbf{w}^T \mathbf{H} \mathbf{w} > 0$.

$$\mathbf{w}^T \mathbf{H} \mathbf{w} = \mathbf{w}^T \Phi^T \mathbf{R} \Phi \mathbf{w}$$

$$\left(\mathbf{R} = \sqrt{\mathbf{R}} \sqrt{\mathbf{R}} \right)$$

$$= \mathbf{w}^T \Phi^T \sqrt{\mathbf{R}} \sqrt{\mathbf{R}} \Phi \mathbf{w}$$

$$\left(\sqrt{\mathbf{R}} = \text{diag}(\sqrt{y_n(1 - y_n)}) \right)$$

$$= (\sqrt{\mathbf{R}} \Phi \mathbf{w})^T (\sqrt{\mathbf{R}} \Phi \mathbf{w})$$

$$= \|\sqrt{\mathbf{R}} \Phi \mathbf{w}\|^2 > 0$$

$$\left(\text{since } \mathbf{w} \neq \mathbf{0} \right)$$